
Software Requirements Specification

for

Digital Classroom – A Classroom Management Platform

Version 1.0 approved

Prepared by Ugesh K

Coimbatore Institute of Technology

09/10/2025

Completed
9/10/25

Table of Contents

Table of Contents	ii
Revision History	ii
1. Introduction.....	1
1.1 Purpose.....	1
1.2 Document Conventions.....	1
1.3 Intended Audience and Reading Suggestions.....	1
1.4 Product Scope	1
1.5 References.....	1
2. Overall Description.....	1
2.1 Product Perspective.....	2
2.2 Product Functions	2
2.3 User Classes and Characteristics	2
2.4 Operating Environment.....	2
2.5 Design and Implementation Constraints.....	3
2.6 User Documentation	3
2.7 Assumptions and Dependencies	3
3. External Interface Requirements	4
3.1 User Interfaces	4
3.2 Hardware Interfaces.....	4
3.3 Software Interfaces	5
3.4 Communications Interfaces	5
4. System Features	6
4.1 System Feature 1.....	6
4.2 System Feature 2.....	6
4.3 System Feature 3.....	7
4.4 System Feature 4.....	8
4.5 System Feature 5.....	9
4.6 System Feature 6.....	9
5. Other Nonfunctional Requirements.....	14
5.1 Performance Requirements.....	14
5.2 Safety Requirements	14
5.3 Security Requirements.....	14
5.4 Software Quality Attributes	15
5.5 Business Rules	16
6. Other Requirements	17
Appendix A: Analysis Models.....	19

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This document describes the functional and non-functional requirements for DigitalClassroom - a web-based classroom management and learning platform designed to simplify and digitize academic workflows. The system enables educational institutions, tuition centers, and independent educators to efficiently manage classes, assignments, attendance, grading, and communication. The goal of this platform is to provide a centralized, user-friendly environment where teachers can manage their teaching activities and students can easily access educational resources, submit assignments, and track their academic performance.

1.2 Document Conventions

This document uses the following conventions

<i>XML</i>	<i>Extention markup language</i>
<i>JSX</i>	<i>JavaScript XML</i>
<i>DV</i>	<i>Daily Value</i>
<i>DB</i>	<i>Database</i>
<i>HTML</i>	<i>Hyper text Markup Language</i>
<i>CSS</i>	<i>Cascading style sheet</i>
<i>JS</i>	<i>Javascript</i>

1.3 Intended Audience and Reading Suggestions

This project is a prototype for the DigitalClassroom system serves as a practical solution for managing classes, assignments, and communication for tuition centers, schools, and colleges. It is intended to benefit teaching staff, administrative teams, and students alike.

1.4 Product Scope

DigitalClassroom aims to unify learning activities such as class management, attendance, assignment handling, and grading into a single digital platform accessible to students, teachers, and administrators.

1.5 References

- IEEE SRS Guidelines
- Krazy tech website

2. Overall Description

2.1 Product Perspective

DigitalClassroom is a standalone platform, deployable over the cloud or intranet, inspired by Google Classroom but tailored to tuition/school environments.

2.2 Product Functions

- Role-based user management
- Class and schedule creation
- Assignment distribution and submission
- Grading system with feedback
- Attendance marking and reports
- Real-time announcements

2.3 User Classes and Characteristics

- **Admin:** Full access to user and system settings
- **Teacher:** Create/manage classes, assignments, attendance
- **Student:** Join/view classes, submit assignments

2.4 Operating Environment

- Web Browser (Chrome, Firefox)
- Frontend: HTML, CSS, Javascript, React
- Backend: Node.js
- Database: Firebase

2.5 Design and Implementation Constraints

- HTTPS for secure communication
- Responsive design (mobile-friendly)
- RESTful APIs
- Frontend Framework: React.js (with optional integration of React Router, Axios, etc.)
- Framework: ReactJS (with Router, Hooks, Firestore)

2.6 User Documentation

- Online Help and Tutorials
- Teacher and Student Guides (PDF)
- FAQs section

2.7 Assumptions and Dependencies

- Reliable internet access for all users
- Basic digital literacy
- Usage of supported browsers and OS

3. External Interface Requirements

3.1 User Interfaces

The system will provide a responsive web-based interface accessible through modern browsers such as Chrome, Edge, and Firefox. It is designed for ease of use by students and teachers with clear navigation, intuitive controls, and consistent layout.

Interfaces include:

- **Login Page:**
Provides secure login and registration for teachers and students using Firebase Authentication (Email/Password and Google Sign-In). Redirects users to dashboards based on their assigned role.
- **Teacher Dashboard:**
Allows teachers to create, edit, and delete classes, post announcements and assignments, view submissions, grade assignments, and upload study materials (PDF, image, or video files).
- **Student Dashboard:**
Enables students to join classes using a class code, view announcements and assignments, submit work, and view grades or teacher feedback.
- **Class Page:**
Displays all announcements, assignments, and materials shared in that class. Supports file previews and submission uploads directly to Firebase Storage.
- **Profile Page:**
Allows users to update their display name and upload or change their profile picture.
- **Notification Area:**
Displays alerts for new announcements, assignments, and grades through real-time Firestore listeners.

The interface is designed to be responsive, mobile-friendly, and accessible, ensuring consistent navigation, buttons, and forms across devices using Tailwind CSS.

3.2 Hardware Interfaces

- **User Devices:**
Any modern desktop or laptop with at least 4 GB RAM and a 2 GHz processor capable of running a modern web browser.
- **Mobile Devices:**
Smartphones and tablets can access the platform using the same web interface. The system layout automatically adapts to smaller screen sizes.
- **Server Infrastructure:**
The application is hosted on Firebase's cloud infrastructure; therefore, no dedicated physical server hardware is required.

3.3 Software Interfaces

- **Operating Systems:**
Compatible with Windows, Linux, and macOS platforms for end users.
- **Frontend:**
Developed using ReactJS (with functional components, Hooks, and React Router v6) and Tailwind CSS for modern, dynamic UI rendering.
- **Backend Services:**
Provided by Firebase, utilizing:
 - Firebase Authentication – for secure user sign-in and role-based access.
 - Cloud Firestore – for real-time NoSQL database operations.
 - Firebase Storage – for file uploads (PDF, DOCX, PNG, MP4, etc.).
- **Database:**
Firestore database structure includes collections such as users, classes, classMembers, posts, and submissions.
- **Authentication:**
Firebase Auth (Email/Password, Google Sign-In). Session management handled automatically by Firebase SDK.
- **File Handling:**
Supports uploads of documents, images, and videos. Each file is validated for size and type before storage.

3.4 Communications Interfaces

- **Network Protocols:**
All communication between the frontend and Firebase backend occurs over secure HTTPS using Firebase SDKs.
- **Real-Time Updates:**
Firestore provides real-time synchronization so that any update made by a teacher (e.g., posting an assignment) is instantly visible to students without refreshing.
- **Hosting and Connectivity:**
The system operates entirely in the cloud using Firebase Hosting and requires a stable internet connection.
- **API Access:**
Firebase's SDKs serve as the communication interface between the client and the backend. Future extensions may include REST APIs for external system integration (e.g., institutional portals or LMS).
- **Error Handling:**
The application includes user-friendly messages for network errors, timeouts, and upload failures.

4. System Features

4.1 Feature 1 – User Authentication & Role Management

Description:

The system provides secure authentication for Teachers and Students using Firebase Authentication. Users can register with Email/Password or Google Sign-In and are redirected to their respective dashboards based on role selection.

Inputs:

Email, Password, Display Name, Role (Teacher/Student)

Outputs:

Successful login and redirection to Teacher or Student Dashboard.

Preconditions:

User must be registered with valid credentials in the Firebase Authentication database.

Postconditions:

User session is created and access is granted to the appropriate dashboard.

Normal Flow:

1. User opens the login/registration page.
2. User enters valid credentials or uses Google Sign-In.
3. System verifies credentials via Firebase Authentication.
4. System retrieves the user role from Firestore.
5. System redirects the user to the correct dashboard (Teacher/Student).

Alternative Flow:

- If invalid credentials are entered, the system displays an error message and prompts for re-entry.
- If the user account is not found, the system suggests registration.

4.2 Feature 2 – Class Creation and Joining

Description:

Teachers can create new classes, and students can join existing classes using a unique class code generated automatically by the system.

Inputs:

Class Name, Section, Class Code (auto-generated), Teacher ID, Student ID.

Outputs:

- For Teacher: Class created successfully message.
- For Student: Joined class confirmation message.

Preconditions:

- Teacher must be logged in with a teacher role.
- Student must be logged in to join classes.

Postconditions:

The new class record is stored in Firestore (classes collection), and members are linked in the classMembers collection.

Normal Flow:

1. Teacher navigates to the dashboard and selects "Create Class."
2. Teacher enters class name and section.
3. System generates a unique class code and stores class details.
4. Student selects "Join Class" and enters class code.
5. System validates the code and adds the student to the class.

Alternative Flow:

- If the entered class code is invalid, the system prompts the student to re-enter.
- If the teacher leaves mandatory fields blank, the system displays a validation error.

4.3 Feature 3 – Announcements and Study Material Upload

Description:

Teachers can post announcements and upload study materials such as PDFs, images, or videos for students in their classes. Students can view these materials in real time.

Inputs:

Announcement text, optional file (PDF, DOCX, PNG, MP4), Class ID.

Outputs:

Material or announcement visible on the class feed for all enrolled students.

Preconditions:

- Teacher must be logged in and have an active class.
- File size and format must meet system limits.

Postconditions:

The uploaded file is stored in Firebase Storage, and metadata is saved in Firestore under the class record.

Normal Flow:

1. Teacher navigates to a class page and clicks "Post Announcement."
2. Teacher writes a message and optionally attaches files.

3. System uploads files to Firebase Storage.
4. Firestore updates class feed with new post data.
5. Students receive a real-time update and can view/download materials.

Alternative Flow:

- If the file exceeds the maximum size limit, the system shows an error message.
- If there is a network interruption, the upload is paused or retried.

4.4 Feature 4 – Assignment Submission and Grading

Description:

Teachers can create assignments and grade student submissions. Students can submit assignments by uploading files and text responses before deadlines.

Inputs:

Assignment title, description, due date, submission file (PDF/DOCX/ZIP), Grade/Feedback.

Outputs:

For Student: Confirmation of successful submission.

For Teacher: Assignment and grade records visible on dashboard.

Preconditions:

- Teacher must have created a class.
- Student must have joined the class.

Postconditions:

Assignments and submissions are recorded in Firestore under respective collections (assignments, submissions).

Normal Flow:

1. Teacher posts an assignment in the class.
2. Student views the assignment on the class page.
3. Student uploads the completed work before the deadline.
4. System stores the submission in Firebase Storage.
5. Teacher reviews submissions and assigns grades with feedback.
6. Student views graded result and feedback in the dashboard.

Alternative Flow:

- If the student uploads after the deadline, the system shows a late submission warning.
- If the uploaded file format is unsupported, the system rejects it.

4.5 Feature 5 – Real-Time Notifications

Description:

The system provides instant notifications for new announcements, assignments, grades, and class updates using Firestore listeners.

Inputs:

Triggered database changes in Firestore collections (posts, assignments, grades).

Outputs:

Popup or visual indication of new updates on the dashboard.

Preconditions:

User must be logged in and have active class memberships.

Postconditions:

The user's dashboard is updated in real-time without manual refresh.

Normal Flow:

1. Teacher posts new material or grades an assignment.
2. Firestore listener detects the change.
3. System pushes real-time update to all class members.
4. Student dashboard shows notification alert.

Alternative Flow:

- If the user is offline, notifications appear once the connection is restored.
- If Firestore listener fails, data will be re-synced on page reload.

4.6 Feature 6 – Profile Management

Description:

Both Teachers and Students can manage their profile information, including display name and profile picture.

Inputs:

Display Name, Profile Picture (image file).

Outputs:

Updated profile displayed across all dashboards and posts.

Preconditions:

User must be logged in with an active account.

Postconditions:

Profile data is updated in both Firebase Authentication and Firestore.

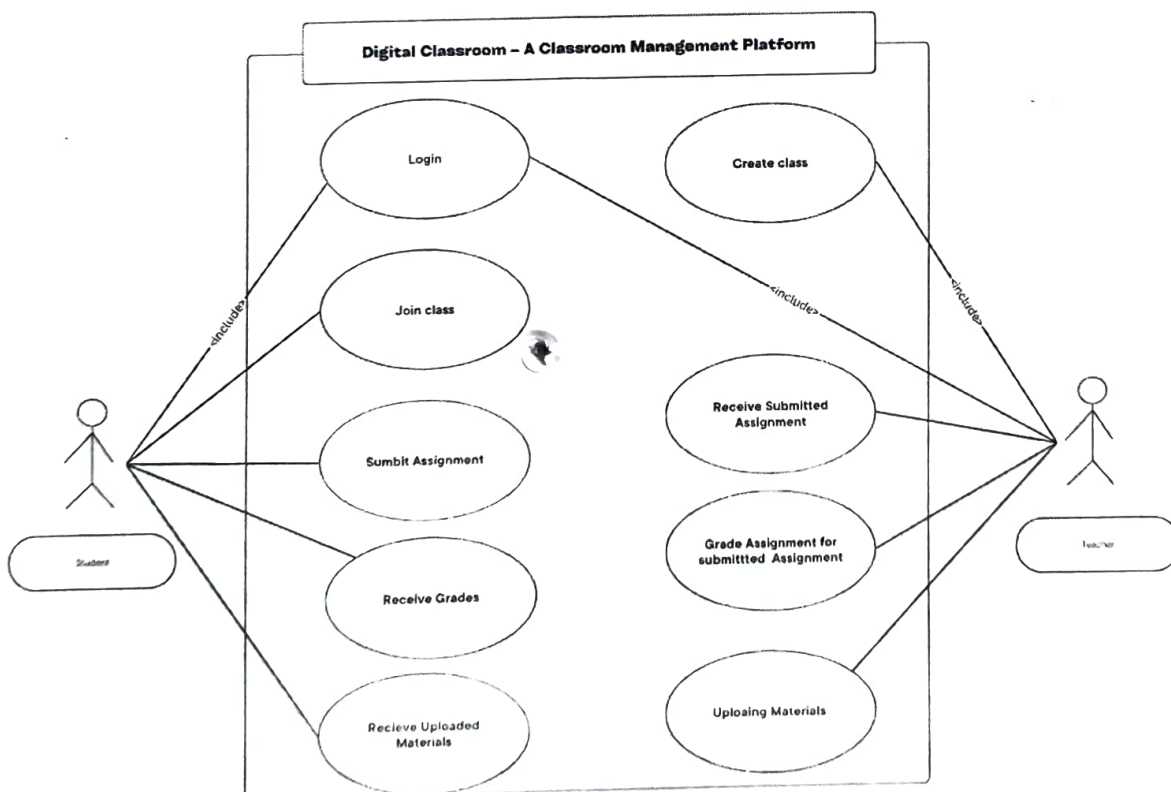
Normal Flow:

1. User opens the Profile page.
2. User edits their name or uploads a new photo.
3. System uploads the image to Firebase Storage.
4. Firestore and Firebase Auth are updated with the new data.
5. Updated profile is reflected across the application.

Alternative Flow:

- If the image exceeds allowed size or format, the system rejects it.
- If upload fails, the system shows an error message.

USE CASE DIAGRAM



Digital Classroom System → Teacher / Student

UC1: Submit Assignment

- **Actor:** Student
- **Secondary Actor:** Teacher

- **Precondition:** Student must be logged in and joined in a class
- **Trigger:** Student selects "Submit Assignment"

Main Success Scenario:

1. Student selects an assignment to submit
2. System validates file format and size
3. System accepts and stores the assignment
4. System notifies teacher of submission
5. System updates database record

Exception Scenarios:

- a) Invalid file format or size exceeded
→ System rejects file and prompts to re-upload
- b) Deadline missed
→ System prevents submission and notifies student
- c) No network/server error
→ System shows error and allows retry

UC2: Grade Assignment

- **Actor:** Teacher
- **Secondary Actor:** Student
- **Precondition:** Student must have submitted the assignment
- **Trigger:** Teacher selects "Grade Submission"

Main Success Scenario:

1. Teacher opens assignment submission
2. Teacher reviews the submission
3. Teacher assigns marks and feedback
4. System saves grade and feedback
5. Student receives grade notification

Exception Scenarios:

- a) Assignment file unreadable
→ System informs teacher and marks submission as problematic

- b) Teacher session timeout
→ System prompts to login again before saving

UC3: Join Class

- **Actor:** Student
- **Precondition:** Student account must be active
- **Trigger:** Student selects "Join Class"

Main Success Scenario:

1. Student enters class code
2. System verifies the code
3. System adds student to the class roster
4. Confirmation message is shown to student

Exception Scenarios:

- a) Invalid class code
→ System rejects code and prompts re-entry
- b) Class already full (if limits exist)
→ System denies join request and displays reason

UC4: Upload Study Materials

- **Actor:** Teacher 
- **Precondition:** Teacher must have created or joined a class
- **Trigger:** Teacher selects "Upload Materials"

Main Success Scenario:

1. Teacher uploads material file (PDF, DOC, PPT)
2. System verifies file format and size
3. System uploads and stores material
4. Students are notified of new material upload

Exception Scenarios:

- a) Unsupported file format
→ System rejects upload
- b) File size too large
→ System rejects upload and prompts resizing

UC5: View Grades

- **Actor:** Student
- **Precondition:** Assignment must have been graded
- **Trigger:** Student selects "View Grades"

Main Success Scenario:

1. Student logs in and accesses gradebook
2. System fetches grades
3. Grades and feedback are displayed to student

Exception Scenarios:

- a) Grades not published yet
→ System informs student
- b) Database error
→ System shows technical error message

UC6: Post Announcements

- **Actor:** Teacher
- **Use Case ID:** UC6
- **Precondition:** Teacher must have an active class
- **Primary Scenario:** Teacher posts an announcement
- **Trigger:** Teacher selects "Post Announcement"

Main Success Scenario:

1. Teacher writes announcement message
2. System posts announcement to class feed
3. Students receive notification

Exception Scenarios:

- a) Network issue
→ System shows failed notification and allows retry

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- The system shall load the dashboard and classroom pages within 3–5 seconds under normal network conditions.
- The system shall support at least 200 concurrent active users (teachers and students) without significant performance degradation.
- Firebase Firestore queries shall return results within 2 seconds for classes containing up to 1,000 records.
- Real-time updates for announcements and submissions shall appear within 2 seconds of posting.
- System downtime for maintenance or updates shall not exceed 2 hours per month.
- The application shall maintain smooth performance across both desktop and mobile browsers.

5.2 Safety Requirements

- All user data (assignments, grades, announcements, and materials) shall be automatically backed up in Firebase Cloud Storage and Firestore to prevent data loss.
- In the event of network or service failure, unsent data shall be queued locally and retried once the connection is restored.
- No operation (e.g., failed file upload) shall corrupt or overwrite existing records in Firestore.
- Access to Firebase Firestore write permissions shall be restricted by security rules to prevent unauthorized modification.
- Teachers and students can only view or modify data that belongs to their own classes.
- In case of accidental deletion, system administrators can recover data from the latest Firebase backup snapshot.

5.3 Security Requirements

- All communications between the client and Firebase backend shall occur over secure HTTPS protocols.
- User authentication shall be managed using Firebase Authentication, which provides encrypted credential handling and session management.
- Role-based access control shall be enforced:

- Students: Join classes, view announcements, submit assignments, and view grades.
- Teachers: Create classes, post materials, view submissions, assign grades.
- Admins (optional): Monitor user activities and manage system-level settings.
- Uploaded files shall be stored securely in Firebase Storage with unique file paths to prevent unauthorized access.
- Firebase Security Rules shall ensure that users can only read or write data relevant to their role and class.
- The system shall log all user sign-ins, class creations, and submissions for auditing and traceability.
- Sensitive information (like email or profile data) shall not be displayed publicly or shared without consent.

5.4 Software Quality Attributes

- **Usability:**
The interface shall be intuitive and consistent, requiring minimal training for first-time users. It shall include clear icons, labels, and consistent navigation for both teacher and student roles.
- **Reliability:**
The system shall achieve at least 99% uptime during operational hours. Firebase's infrastructure ensures data durability and fault tolerance.
- **Scalability:**
The system shall scale automatically with increased users or classes without manual server configuration.
- **Maintainability:**
The ReactJS component-based architecture and Firestore's flexible schema shall allow easy maintenance, feature updates, and debugging.
- **Portability:**
The web app shall be compatible across all major browsers — Chrome, Edge, Firefox, and Safari — and on multiple devices (laptops, tablets, and smartphones).
- **Accessibility:**
The design shall follow WCAG 2.1 guidelines, ensuring that visually impaired users can access the system using assistive technologies.

5.5 Business Rules

- Only authenticated users may access the system features.
- Teachers can manage only the classes they created.
- Students can join multiple classes but cannot modify teacher content.
- Each class code shall be unique and generated automatically by the system.
- Assignment submissions cannot be altered after the due date unless the teacher reopens the assignment.
- All user data, including grades and feedback, shall be stored securely and preserved for a minimum of 5 years.
- Teachers have the final authority to grade and modify assignment results.

6. Other Requirements

6.1 Database Requirements

- The system shall use Firebase Firestore (NoSQL) for storing user, class, and assignment data.
- Data structure shall follow normalized collections:
 - users — Stores profile and role data.
 - classes — Stores class details and teacher IDs.
 - classMembers — Links students to classes.
 - posts — Stores announcements and assignments.
 - submissions — Stores assignment submissions and grades.
- Firestore shall automatically index queries for fast retrieval.
- Data redundancy shall be avoided using document references instead of duplication.
- Daily automatic backups shall be managed by Firebase.

6.2 Internationalization Requirements

- The system shall support English as the default language.
- The architecture shall allow future addition of other languages (e.g., Tamil, Hindi, French) through localization files or Firebase Remote Config.
- All date/time formats shall follow the user's local timezone (IST by default).

6.3 Legal Requirements

- The system shall comply with applicable data protection and privacy laws, including handling of educational data in accordance with institutional policies.
- Teachers and students must provide consent before their personal data (name, email, photo) is stored or shared.
- Any copyrighted material uploaded by teachers (e.g., books, notes, videos) shall include appropriate attribution and must not violate copyright regulations.
- User data shall not be sold, shared, or disclosed to third parties without explicit consent.

6.4 Reuse Objectives

- Core modules such as Authentication, Class Management, and File Upload shall be modular and reusable across other educational platforms.
- The Firebase setup and ReactJS components shall be reusable for similar classroom or LMS (Learning Management System) applications.
- The project architecture shall support future integration with APIs such as Google Drive, Zoom, or Microsoft Teams for enhanced learning capabilities.

6.5 Documentation Requirements

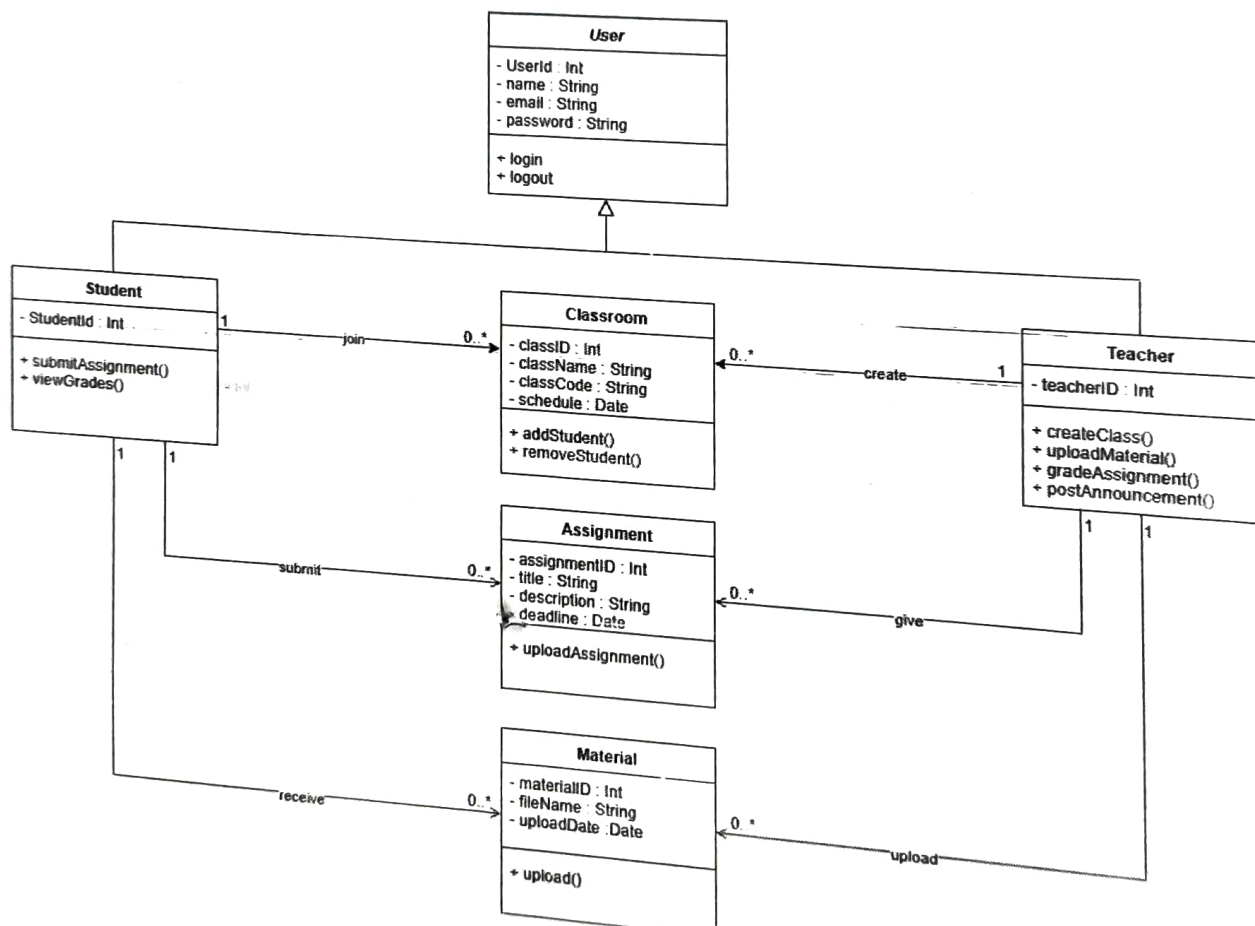
- A User Manual shall be provided for both Teachers and Students, explaining how to create/join classes, upload materials, and submit assignments.
- A Developer Guide shall be maintained detailing component structure, Firebase collections, and deployment instructions.
- API documentation for Firebase rules, collections, and endpoints shall be maintained using inline comments and versioned Git commits.
- Tutorials, FAQ pages, and troubleshooting guides shall be updated with every major release.

6.6 Future Enhancements

- Integration of AI-based recommendation system to suggest materials or assignments to students based on performance.
- Dark Mode toggle for better accessibility and user preference.
- Bulk ZIP download of all student submissions for teachers.
- Push Notifications via Firebase Cloud Messaging for instant alerts.
- Offline mode allowing users to view recent classes and posts even without internet connectivity.
- Integration with third-party LMS systems or institutional portals for extended features.

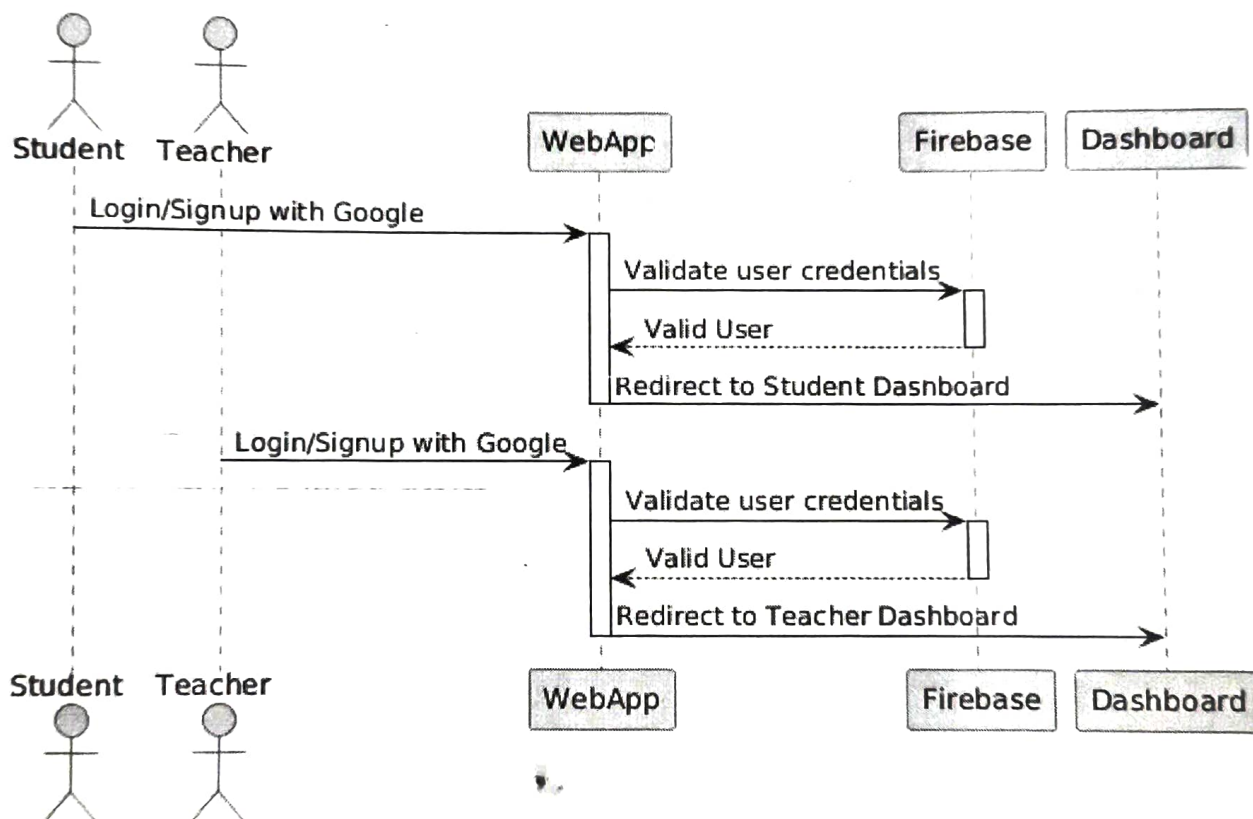
Appendix : Analysis Of Models

Class Diagram

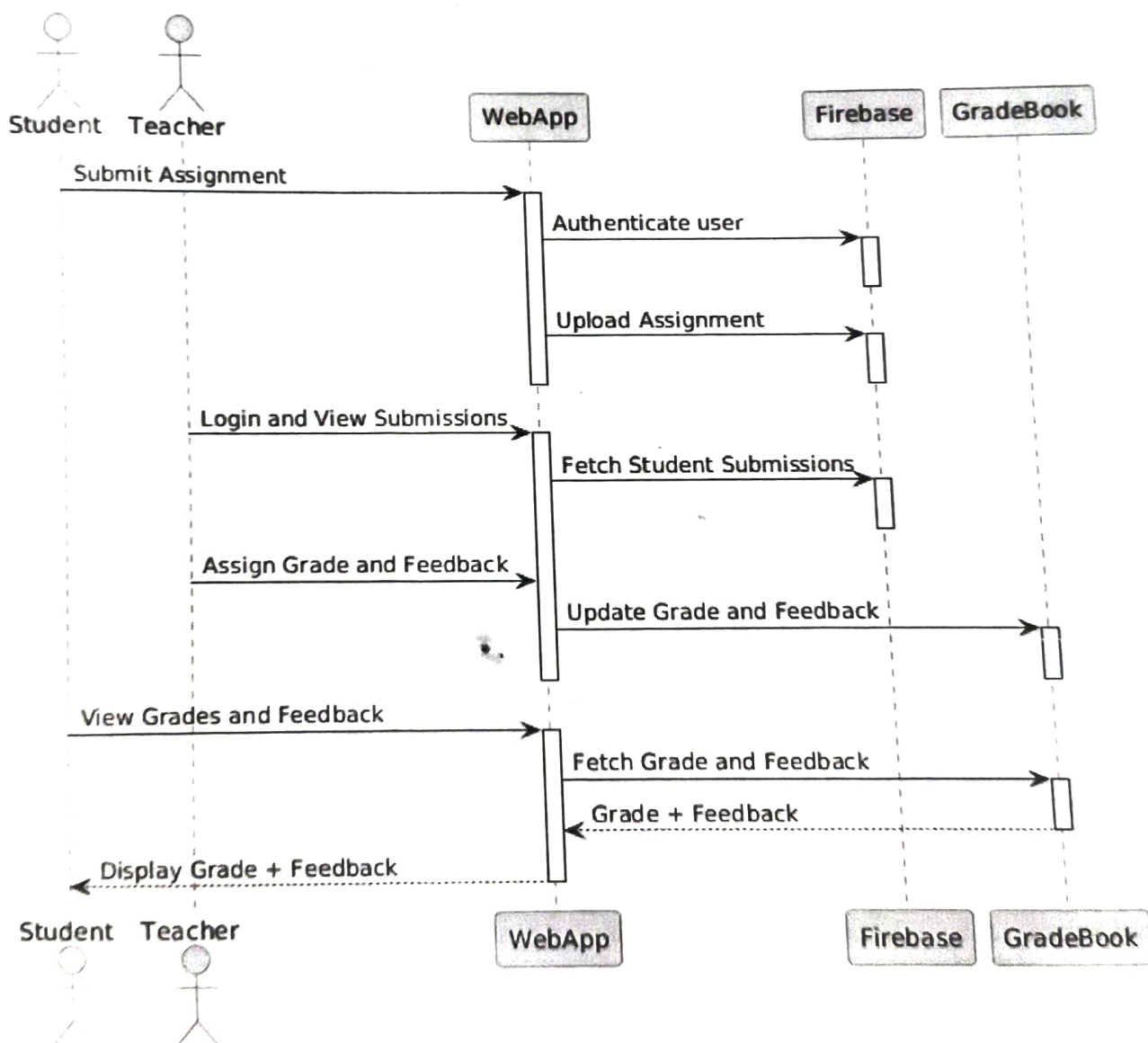


Sequence Diagram

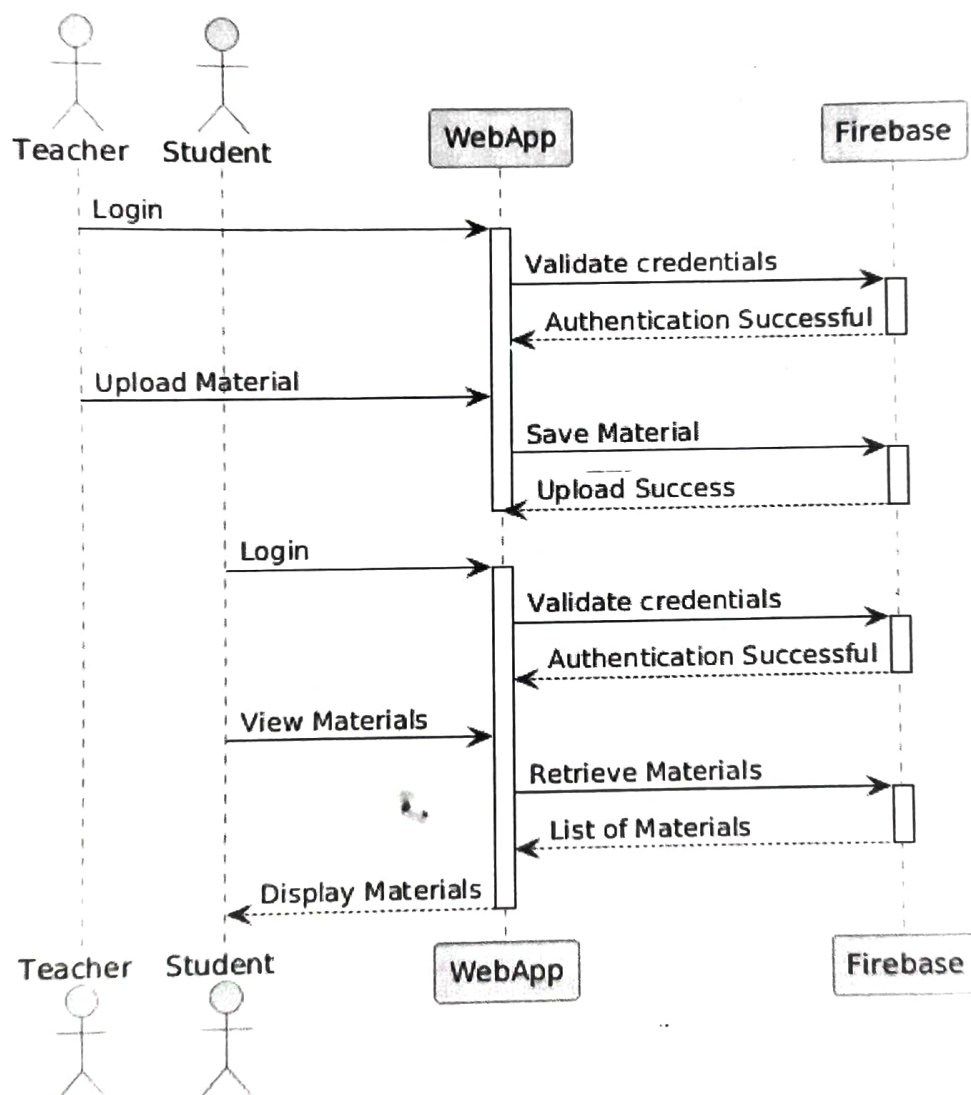
Login and Dashboard Access



Assignment Submission and Grading

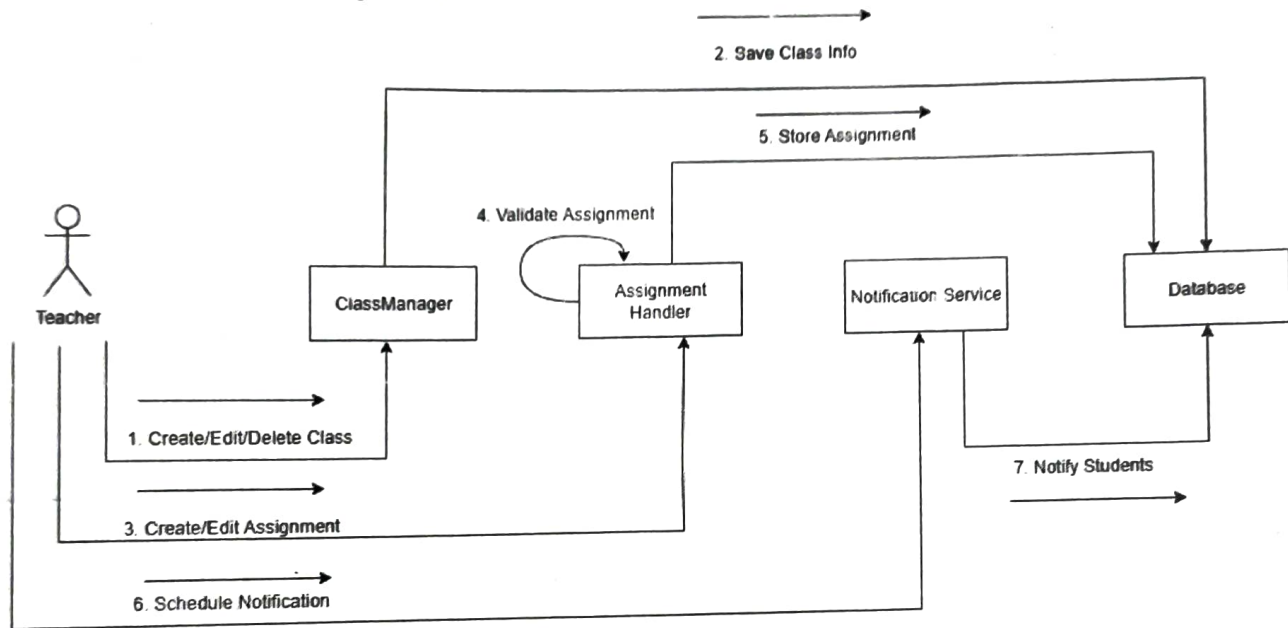


Material Upload and Access

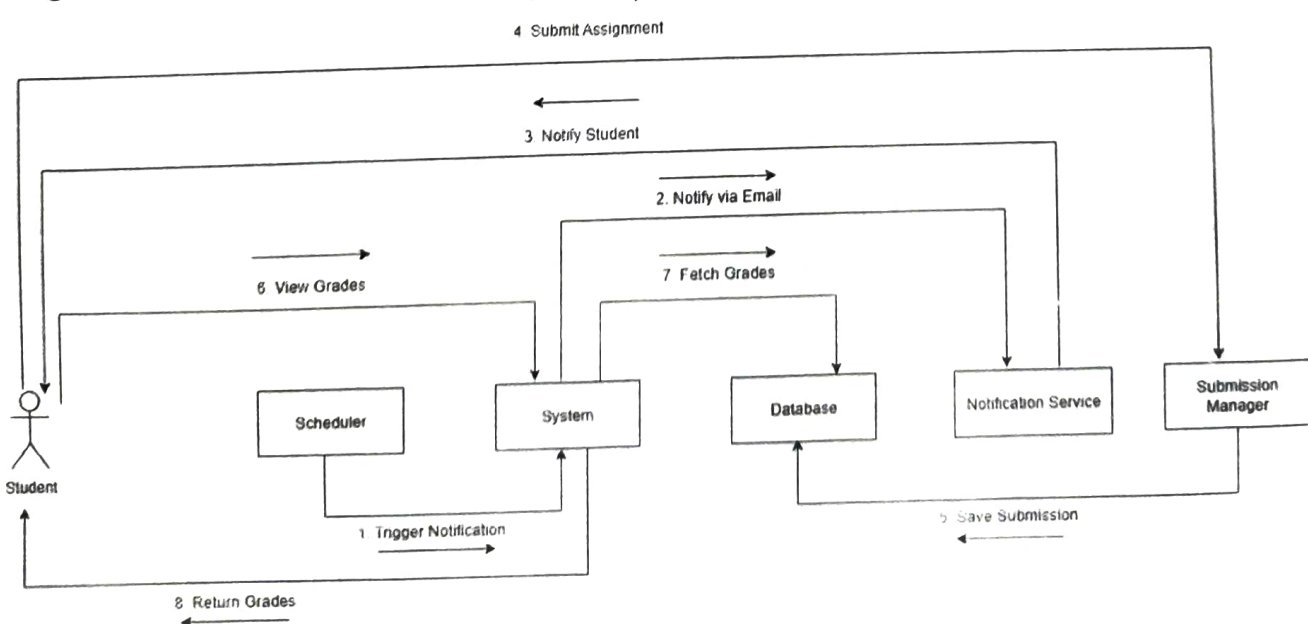


Collaboration Diagram

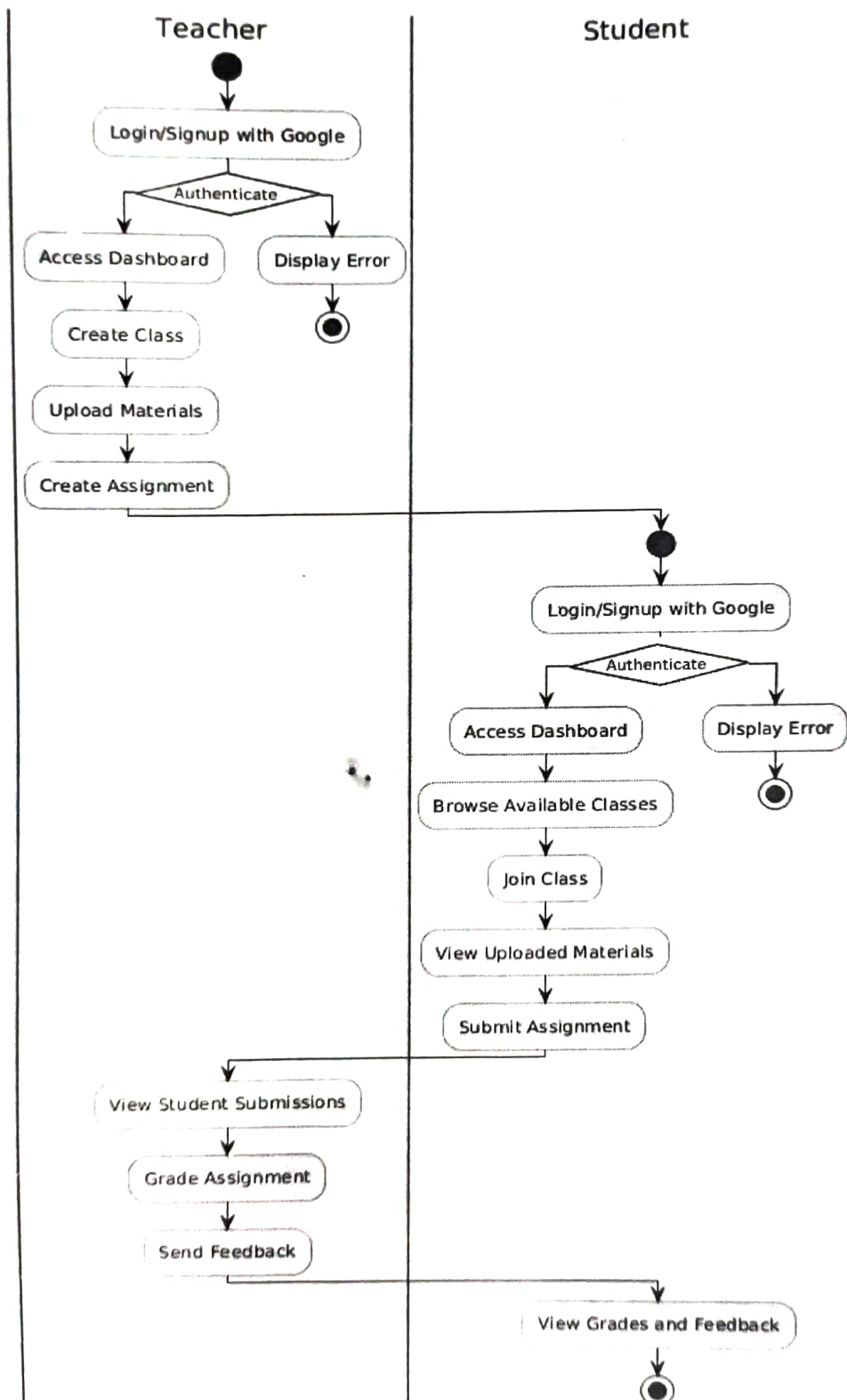
Class and Assignment Management



Assignment Submission & Notification Update

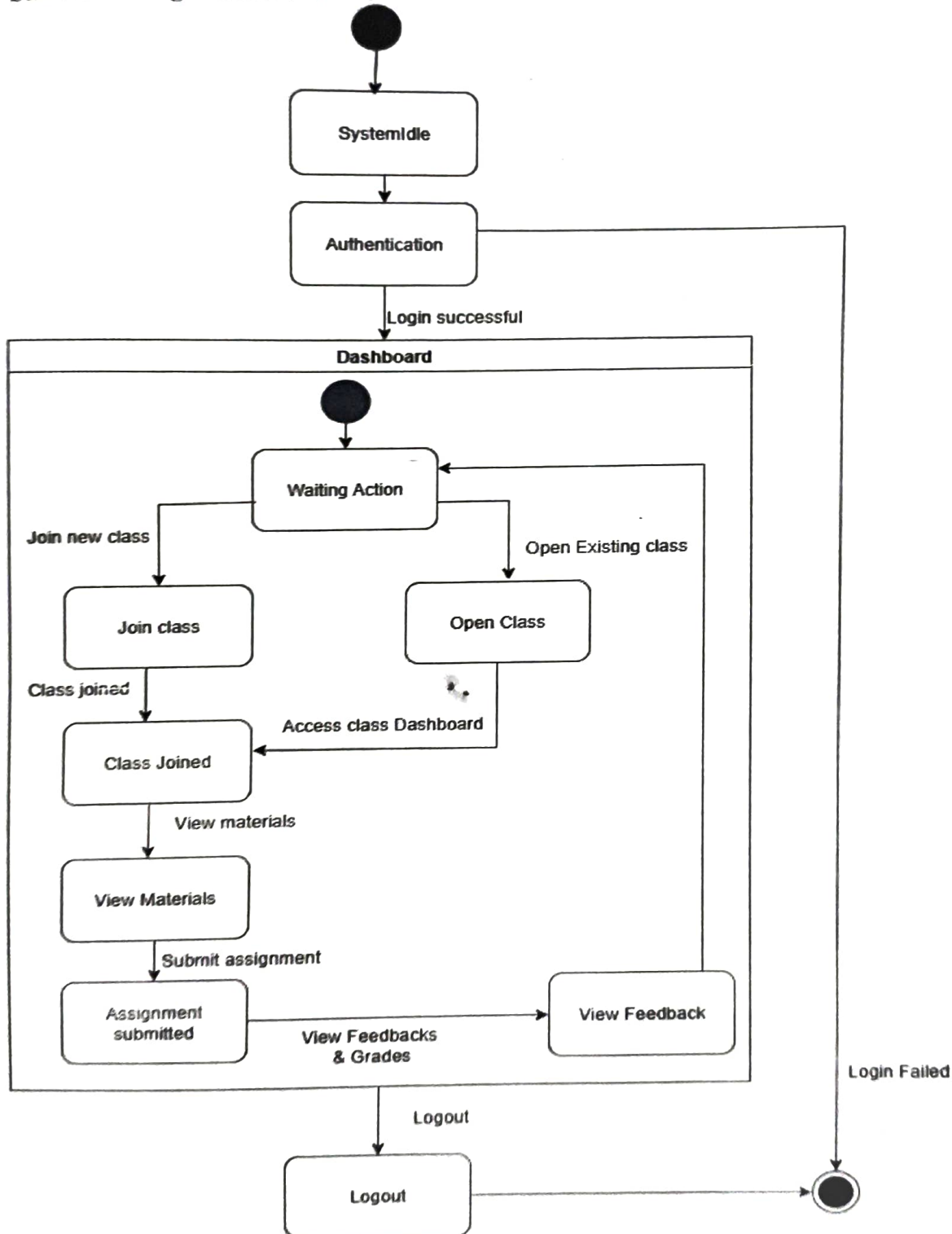


Activity Diagram

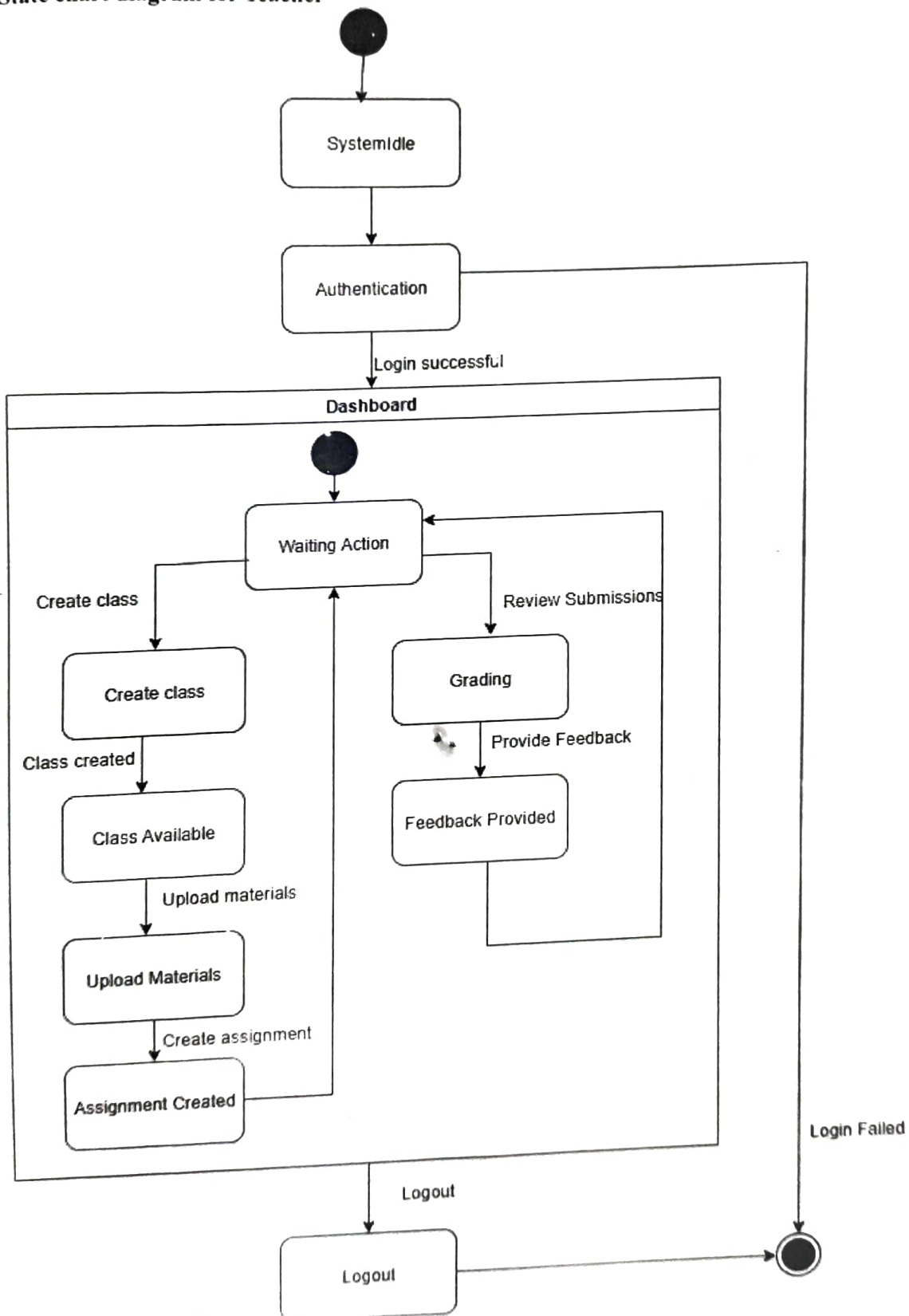


State Chart Diagram

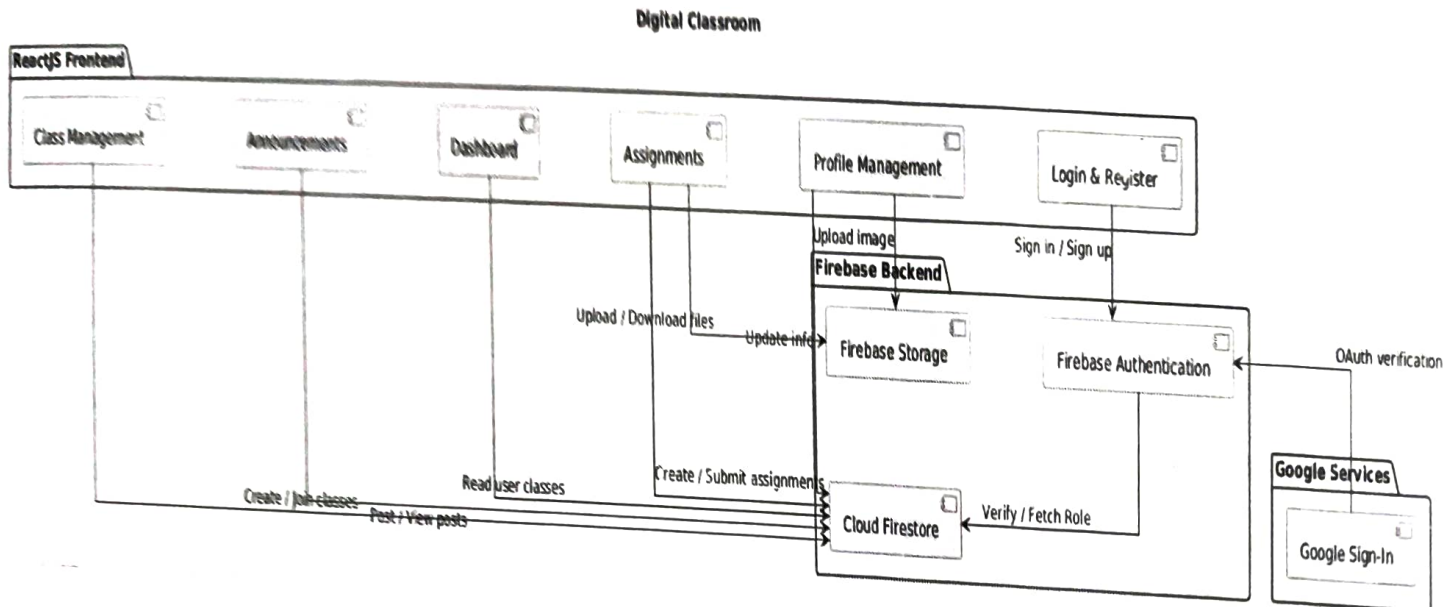
State chart diagram for Students



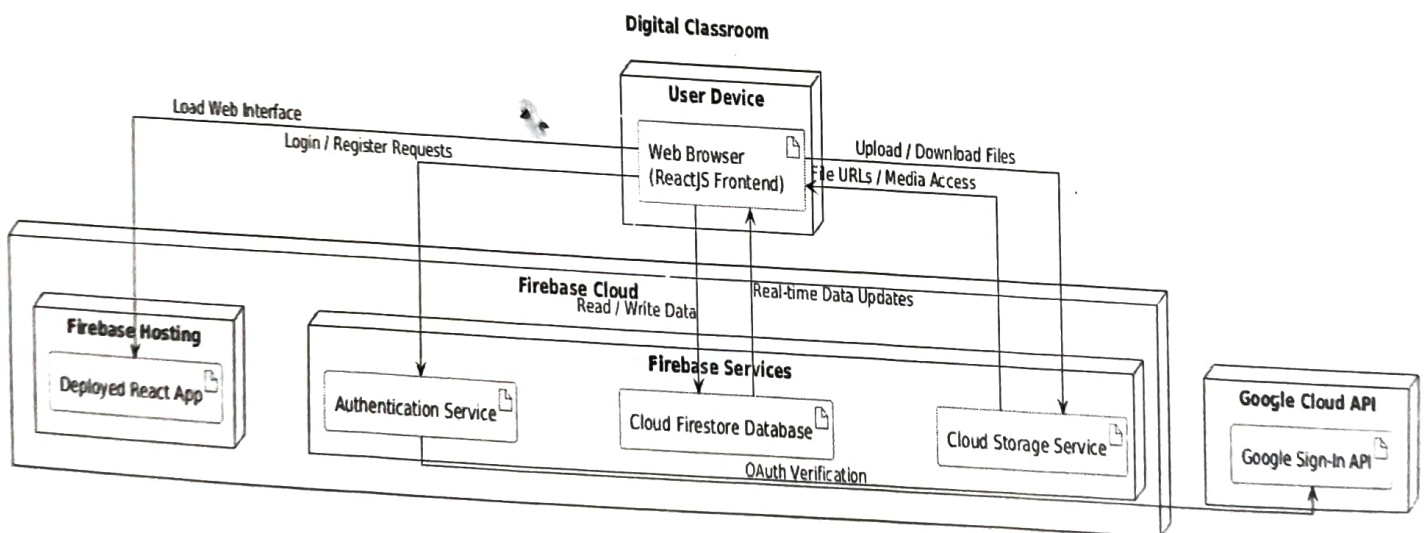
State chart diagram for Teacher



Component Diagram



Deployment Diagram



Source Code

App.js

```
import React from 'react';
import { Routes, Route, Navigate } from 'react-router-dom';
import Login from './pages/Login';
import Register from './pages/Register';
import Dashboard from './pages/Dashboard';
import ClassPage from './pages/ClassPage';
import Profile from './pages/Profile';
import { AuthProvider, useAuth } from './contexts/AuthContext';
import Navbar from './components/Navbar';

function ProtectedRoute({ children }) {
  const { user, loading } = useAuth();
  if (loading) return <div className="p-4">Loading...</div>;
  if (!user) return <Navigate to="/login" replace />;
  return children;
}

export default function App() {
  return (
    <AuthProvider>
      <div className="min-h-screen bg-gray-50">
        <Navbar />
        <div className="container mx-auto p-4">
          <Routes>
            <Route path="/" element={<Navigate to="/dashboard" replace />} />
            <Route path="/login" element={<Login />} />
            <Route path="/register" element={<Register />} />
            <Route path="/dashboard" element={
              <ProtectedRoute><Dashboard /></ProtectedRoute>
            } />
            <Route path="/class/:id" element={
              <ProtectedRoute><ClassPage /></ProtectedRoute>
            } />
            <Route path="/profile" element={
              <ProtectedRoute><Profile /></ProtectedRoute>
            } />
            <Route path="*" element={<div>Not Found</div>} />
          </Routes>
        </div>
      </div>
    </AuthProvider>
  );
}
```

Firebase.js

```
import { initializeApp } from "firebase/app";
import { getAuth, GoogleAuthProvider, signInWithPopup } from "firebase/auth";
import { getFirestore, serverTimestamp } from "firebase/firestore";
import { getStorage } from "firebase/storage";
```

```
const firebaseConfig = {
  apiKey: "AIzaSyAu3OtxclQ_Q58ne6vZodbJRD8o_tluYAw",
  authDomain: "digital-classroom-5ca62.firebaseio.com",
  projectId: "digital-classroom-5ca62",
  storageBucket: "digital-classroom-5ca62.firebaseio.com",
  messagingSenderId: "466660725943",
  appId: "1:466660725943:web:910166325e2d780a616932",
  measurementId: "G-BC5EVMCG33"
};
```

```
const app = initializeApp(firebaseConfig);
```

```
export const auth = getAuth(app);
export const googleProvider = new GoogleAuthProvider();
export const db = getFirestore(app);
export const storage = getStorage(app);
```

```
export { serverTimestamp, signInWithPopup };
```

Login.js

```
import React, { useState } from 'react';
import { useNavigate, Link } from 'react-router-dom';
import { useAuth } from '../contexts/AuthContext';
```

```
export default function Login() {
  const { login, loginWithGoogle } = useAuth();
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const nav = useNavigate();
```

```
const submit = async (e) => {
  e.preventDefault();
  try {
    await login(email, password);
    nav('/dashboard');
  } catch(err) {
    setError(err.message);
  }
};
```

```
const google = async () => {
  try {
    await loginWithGoogle();
    nav('/dashboard');
  } catch(err) { setError(err.message); }
};
```

```
return (
  <div className="max-w-md mx-auto bg-white p-6 rounded shadow">
    <h2 className="text-2xl font-semibold mb-4">Login</h2>
    {error && <div className="text-red-600 mb-2">{error}</div>}
    <form onSubmit={submit} className="space-y-3">
```



```

    <input className="w-full p-2 border rounded" placeholder="Email" value={email}
onChange={e=>setEmail(e.target.value)} />
    <input type="password" className="w-full p-2 border rounded" placeholder="Password"
value={password} onChange={e=>setPassword(e.target.value)} />
    <button className="w-full bg-blue-600 text-white p-2 rounded">Login</button>
  </form>
  <button onClick={google} className="w-full mt-3 border p-2 rounded">Sign in with Google</button>
  <p className="mt-3 text-sm">Don't have an account? <Link className="text-blue-600"
to="/register">Register</Link></p>
</div>
);
}

```

Profile.js

```

import React, { useState } from 'react';
import { useAuth } from '../contexts/AuthContext';
import { storage, db } from '../firebase';
import { ref, uploadBytesResumable, getDownloadURL } from 'firebase/storage';
import { doc, updateDoc } from 'firebase/firestore';
import { updateProfile } from 'firebase/auth';
import { auth } from '../firebase';

export default function Profile() {
  const { user, profile } = useAuth();
  const [file, setFile] = useState(null);
  const [displayName, setDisplayName] = useState(profile?.displayName || "");
  const [msg, setMsg] = useState("");

  const upload = async () => {
    if (!file) return setMsg('Select an image');
    const sref = ref(storage, `avatars/${user.uid}/${file.name}`);
    const task = uploadBytesResumable(sref, file);
    await task;
    const url = await getDownloadURL(sref);
    // update auth profile and firestore
    await updateProfile(auth.currentUser, { photoURL: url, displayName });
    const docRef = doc(db, 'users', user.uid);
    await updateDoc(docRef, { photoURL: url, displayName });
    setMsg('Profile updated');
  };

  return (
    <div className="max-w-md bg-white p-4 rounded shadow">
      <h2 className="text-xl font-semibold mb-2">Profile</h2>
      <div className="mb-2">Name</div>
      <input className="w-full p-2 border rounded mb-2" value={displayName}
onChange={e=>setDisplayName(e.target.value)} />
      <div className="mb-2">Profile picture</div>
      <input type="file" onChange={e=>setFile(e.target.files[0])} />
      <div className="flex gap-2 mt-3">
        <button className="px-3 py-1 border rounded" onClick={()=>setMsg("")}>Cancel</button>
        <button className="px-3 py-1 bg-blue-600 text-white rounded" onClick={upload}>Save</button>
      </div>
      {msg && <div className="mt-2 text-green-700">{msg}</div>}
    </div>
  );
}

```

```

    </div>
  );
}

```

Register.js

```

import React, { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../contexts/AuthContext';

export default function Register() {
  const { register } = useAuth();
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [displayName, setDisplayName] = useState("");
  const [role, setRole] = useState('student');
  const [err, setErr] = useState("");
  const nav = useNavigate();

  const submit = async (e) => {
    e.preventDefault();
    try {
      await register(email, password, displayName, role);
      nav('/dashboard');
    } catch (err) {
      setErr(err.message);
    }
  };

  return (
    <div className="max-w-md mx-auto bg-white p-6 rounded shadow">
      <h2 className="text-2xl font-semibold mb-4">Register</h2>
      {err && <div className="text-red-600">{err}</div>}
      <form onSubmit={submit} className="space-y-3">
        <input className="w-full p-2 border rounded" placeholder="Full name" value={displayName}
onChange={e=>setDisplayName(e.target.value)} />
        <input className="w-full p-2 border rounded" placeholder="Email" value={email}
onChange={e=>setEmail(e.target.value)} />
        <input type="password" className="w-full p-2 border rounded" placeholder="Password"
value={password} onChange={e=>setPassword(e.target.value)} />
        <div>
          <label className="mr-3"><input type="radio" checked={role==='student'}
onChange={()=>setRole('student')} /> Student</label>
          <label><input type="radio" checked={role==='teacher'} onChange={()=>setRole('teacher')} />
Teacher</label>
        </div>
        <button className="w-full bg-green-600 text-white p-2 rounded">Register</button>
      </form>
    </div>
  );
}

```


Dashboard.js

```
import React, { useEffect, useState } from 'react';
import { useAuth } from '../contexts/AuthContext';
import { collection, query, where, onSnapshot, getDoc, doc } from 'firebase/firestore';
import { db } from '../firebase';
import ClassCard from '../components/ClassCard';
import CreateClassModal from '../components/CreateClassModal';
import JoinClassModal from '../components/JoinClassModal';
```

```
export default function Dashboard() {
  const { profile, user } = useAuth();
  const [classes, setClasses] = useState([]);
  const [loading, setLoading] = useState(true);
```

```
  useEffect(() => {
    if (!profile) return;
```

```
    if (profile.role === 'teacher') {
      // Teacher: classes created by them
      const q = query(collection(db, 'classes'), where('createdBy', '==', user.uid));
      const unsub = onSnapshot(q, snap => {
        setClasses(snap.docs.map(d => ({ id: d.id, ...d.data() })));
        setLoading(false);
      });
      return unsub;
```

```
    } else {
      // Student: fetch classMembers, then fetch each class doc
      const q = query(collection(db, 'classMembers'), where('userId', '==', user.uid));
      const unsub = onSnapshot(q, async snap => {
        const classItems = await Promise.all(
          snap.docs.map(async d => {
            const data = d.data();
            if (!data.classId) return null;
            const clsSnap = await getDoc(doc(db, 'classes', data.classId));
            if (!clsSnap.exists()) return null;
            return { id: clsSnap.id, ...clsSnap.data() };
          })
        );
        setClasses(classItems.filter(Boolean));
        setLoading(false);
      });
      return unsub;
```

```
    }, [profile, user]);
```

```
    if (!profile) return <div>Loading profile...</div>;
```


Dashboard.js

```

import React, { useEffect, useState } from 'react';
import { useAuth } from '../contexts/AuthContext';
import { collection, query, where, onSnapshot, getDoc, doc } from 'firebase/firestore';
import { db } from '../firebase';
import ClassCard from '../components/ClassCard';
import CreateClassModal from '../components/CreateClassModal';
import JoinClassModal from '../components/JoinClassModal';

export default function Dashboard() {
  const { profile, user } = useAuth();
  const [classes, setClasses] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    if (!profile) return;

    if (profile.role === 'teacher') {
      // Teacher: classes created by them
      const q = query(collection(db, 'classes'), where('createdBy', '==', user.uid));
      const unsub = onSnapshot(q, snap => {
        setClasses(snap.docs.map(d => ({ id: d.id, ...d.data() })));
        setLoading(false);
      });
      return unsub;
    } else {
      // Student: fetch classMembers, then fetch each class doc
      const q = query(collection(db, 'classMembers'), where('userId', '==', user.uid));
      const unsub = onSnapshot(q, async snap => {
        const classItems = await Promise.all(
          snap.docs.map(async d => {
            const data = d.data();
            if (!data.classId) return null;
            const clsSnap = await getDoc(doc(db, 'classes', data.classId));
            if (!clsSnap.exists()) return null;
            return { id: clsSnap.id, ...clsSnap.data() };
          })
        );
        setClasses(classItems.filter(Boolean));
        setLoading(false);
      });
      return unsub;
    }
  }, [profile, user]);

  if (!profile) return <div>Loading profile...</div>;

  return (
    <div>
      <div className="flex items-center justify-between mb-4">
        <h1 className="text-2xl font-bold">Welcome, {profile.displayName || 'User'} </h1>
      </div>
      <div className="flex gap-2">
        {profile.role === 'teacher' && <CreateClassModal />}
        <JoinClassModal />
      </div>
    </div>
  );
}

```

```

    </div>
  </div>

  <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
    {loading && <div>Loading classes...</div>}
    {!loading && classes.length === 0 && <div>No classes yet.</div>}
    {classes.map(c => (
      <ClassCard key={c.id} data={c} myRole={profile.role} />
    ))}
  </div>
</div>
);
}

```

Classpage.js

```

import React, { useEffect, useState } from 'react';
import { useParams } from 'react-router-dom';
import { doc, getDoc, collection, query, where, onSnapshot, addDoc, serverTimestamp } from
'firebase/firestore';
import { db, storage } from '../firebase';
import { useAuth } from '../contexts/AuthContext';
import { ref, uploadBytesResumable, getDownloadURL } from 'firebase/storage';

export default function ClassPage() {
  const { id } = useParams();
  const [classDoc, setClassDoc] = useState(null);
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(true);
  const { profile, user } = useAuth();
  const [content, setContent] = useState("");
  const [file, setFile] = useState(null);
  const [uploading, setUploading] = useState(false);

  useEffect(() => {
    const clsRef = doc(db, 'classes', id);
    getDoc(clsRef).then(snap => setClassDoc({ id: snap.id, ...snap.data() }));
    const q = query(collection(db, 'posts'), where('classId', '==', id));
    const unsub = onSnapshot(q, snap => {
      setPosts(snap.docs.map(d=>({ id: d.id, ...d.data() })));
      setLoading(false);
    });
    return unsub;
  }, [id]);

  const handleFileChange = (e) => {
    const selectedFile = e.target.files[0];
    if (selectedFile) {
      // Check file size (max 10MB)
      const maxSize = 10 * 1024 * 1024; // 10MB in bytes
      if (selectedFile.size > maxSize) {
        alert('File size must be less than 10MB');
        e.target.value = "";
        return;
      }
    }
  }
}

```

```

// Check file type (optional - allow common document and image types)
const allowedTypes = [
  'application/pdf',
  'application/msword',
  'application/vnd.openxmlformats-officedocument.wordprocessingml.document',
  'application/vnd.ms-excel',
  'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet',
  'application/vnd.ms-powerpoint',
  'application/vnd.openxmlformats-officedocument.presentationml.presentation',
  'text/plain',
  'image/jpeg',
  'image/png',
  'image/gif',
  'image/webp'
];

if (!allowedTypes.includes(selectedFile.type)) {
  alert('File type not supported. Please upload PDF, Word, Excel, PowerPoint, text, or image files.');
```

e.target.value = "";

```

  return;
}

setFile(selectedFile);
}
};

const postAnnouncement = async () => {
  if (!content.trim() && !file) {
    alert('Please enter content or select a file');
    return;
  }

  setUploading(true);
  try {
    let fileURL = "";
    if (file) {
      console.log('Starting file upload...', file.name);
      const sref = ref(storage, `classFiles/${id}/${Date.now()}_${file.name}`);

      try {
        const uploadTask = uploadBytesResumable(sref, file);

        // Wait for upload to complete
        await new Promise((resolve, reject) => {
          uploadTask.on(
            'state_changed',
            (snapshot) => {
              const progress = (snapshot.bytesTransferred / snapshot.totalBytes) * 100;
              console.log('Upload is ' + progress + "% done");
            },
            (error) => {
              console.error('Upload failed:', error);
              reject(error);
            }
          );
        });
      }
    }
  }
}

```

```

    }
    console.log('Upload completed successfully');
    resolve();
  }
}

fileURL = await getDownloadURL(ref);
console.log('File URL obtained', fileURL);
} catch (uploadError) {
  console.error('File upload error', uploadError);
  alert('Failed to upload file. Please try again.');
```

return;

```

}

await addDoc(collection(db, 'posts'), {
  classId: id,
  authorId: user.uid,
  authorName: profile.displayName || '',
  content,
  fileURL,
  fileName: file ? file.name : '',
  createdAt: serverTimestamp(),
  type: 'announcement'
});

setContent('');
setFile(null);
// Reset file input
const fileInput = document.querySelector('input[type="file"]');
if (fileInput) fileInput.value = '';

console.log('Post created successfully');
} catch (error) {
  console.error('Error posting announcement:', error);
  alert('Failed to post announcement. Please try again.');
```

finally {

```

  setUploading(false);
}

if (!classDoc) return <div>Loading class...</div>;

return (
  <div>
    <div className="bg-white p-4 rounded shadow mb-4">
      <h2 className="text-xl font-semibold">{classDoc.name}</h2>
      <p className="text-sm text-gray-500">Code: <strong>{classDoc.code}</strong></p>
    </div>

    {profile?.role === 'teacher' && (
      <div className="bg-white p-4 rounded shadow mb-4">
```



```

<h3 className="font-semibold mb-2">Create Announcement / Assignment</h3>
<textarea
  className="w-full p-2 border rounded mb-2"
  value={content}
  onChange={e=>setContent(e.target.value)}
  placeholder="Write announcement or assignment details"
  rows="3"
></textarea>
<div className="mb-2">
  <input
    type="file"
    onChange={handleFileChange}
    className="block w-full text-sm text-gray-500 file:mr-4 file:py-2 file:px-4 file:rounded-full
file:border-0 file:text-sm file:font-semibold file:bg-blue-50 file:text-blue-700 hover:file:bg-blue-100"
  />
  {file && (
    <p className="text-sm text-gray-600 mt-1">
      Selected: {file.name} ({(file.size / 1024 / 1024).toFixed(2)} MB)
    </p>
  )}
</div>
<div className="flex gap-2 justify-end mt-2">
  <button
    className="px-3 py-1 border rounded"
    onClick={() => {
      setContent("");
      setFile(null);
      const fileInput = document.querySelector('input[type="file"]');
      if (fileInput) fileInput.value = "";
    }}
    disabled={uploading}
  >
    Cancel
  </button>
  <button
    className={`px-3 py-1 rounded text-white ${uploading ? 'bg-gray-400' : 'bg-blue-600 hover:bg-
blue-700'}`}
    onClick={postAnnouncement}
    disabled={uploading}
  >
    {uploading ? 'Posting...' : 'Post'}
  </button>
</div>
</div>
)}

<div className="space-y-3">
  {loading && <div>Loading posts...</div>}
  {posts.map(p => (
    <div key={p.id} className="bg-white p-3 rounded shadow">
      <div className="text-sm text-gray-600">{p.authorName} •
      {p.createdAt?.toDate?.()?.toLocaleString?.() || ""}</div>
      <div className="mt-2">{p.content}</div>
      {p.fileURL && (

```

```

<div className="mt-2">
  <a
    href={p.fileURL}
    target="_blank"
    rel="noreferrer"
    className="text-blue-600 hover:text-blue-800 underline"
  >
    {p.fileName || 'Attached file'}
  </a>
</div>
)}
</div>
)}}
</div>
</div>
);
}

```

CreateClass.js

```

import React, { useState } from 'react';
import { useAuth } from '../contexts/AuthContext';
import { collection, addDoc, serverTimestamp } from 'firebase/firestore';
import { db } from '../firebase';
import { v4 as uuidv4 } from 'uuid';

export default function CreateClassModal() {
  const { profile, user } = useAuth();
  const [open, setOpen] = useState(false);
  const [name, setName] = useState("");
  const [section, setSection] = useState("");
  const [err, setErr] = useState("");

  const createClass = async () => {
    if (!name) { setErr('Name required'); return; }
    const code = Math.random().toString(36).substring(2,8).toUpperCase();
    await addDoc(collection(db, 'classes'), {
      name, section, code, createdBy: user.uid, createdAt: serverTimestamp()
    });
    setOpen(false); setName(""); setSection("");
  };

  if (profile?.role !== 'teacher') return null;

  return (
    <
      <button onClick={()=>setOpen(true)} className="px-3 py-1 bg-blue-600 text-white rounded">Create
    Class</button>
      {open && (
        <div className="fixed inset-0 bg-black/40 flex items-center justify-center">
          <div className="bg-white p-4 rounded w-96">
            <h3 className="font-semibold mb-2">Create Class</h3>
            {err && <div className="text-red-600">{err}</div>}
            <input className="w-full p-2 border rounded mb-2" placeholder="Class name" value={name}
              onChange={e=>setName(e.target.value)} />
          </div>
        </div>
      )}
    </
  );
}

```

```

    <input className="w-full p-2 border rounded mb-2" placeholder="Section/Subject" value={section}
    onChange={e=>setSection(e.target.value)} />
    <div className="flex gap-2 justify-end">
      <button className="px-3 py-1 border rounded" onClick={()=>setOpen(false)}>Cancel</button>
      <button className="px-3 py-1 bg-green-600 text-white rounded"
      onClick={createClass}>Creat</button>
    </div>
  </div>
</div>
  )}
</>
);
}

```

JoinClass.js

```
import React, { useState } from 'react';
import { collection, query, where, getDocs, addDoc, doc } from 'firebase/firestore';
import { db } from '../firebase';
import { useAuth } from '../contexts/AuthContext';
```

```
export default function JoinClassModal() {
  const [open, setOpen] = useState(false);
  const [code, setCode] = useState("");
  const [msg, setMsg] = useState("");
  const { user } = useAuth();
```

```
const join = async () => {
  setMsg("");
  if (!code) return setMsg('Enter class code');
  const q = query(collection(db, 'classes'), where('code', '==', code));
  const snap = await getDocs(q);
  if (snap.empty) return setMsg('Class not found');
  const cls = snap.docs[0];
}
```

```
await addDoc(collection(db, 'classMembers'), {
  classId: cls.id,
  classRef: doc(db, 'classes', cls.id),
  userId: user.uid,
  joinedAt: new Date()
});
setMsg('Joined!');
setOpen(false);
setCode("");
};
```

```
return (
  <button onClick={()=>setOpen(true)} className="px-3 py-1 border rounded">Join Class</button>
  {open && (
    <div className="fixed inset-0 bg-black/40 flex items-center justify-center">
      <div className="bg-white p-4 rounded w-96">
        <h3 className="font-semibold mb-2">Join Class with Code</h3>
        <h3 className="font-semibold mb-2">Join Class with Code</h3>
        {msg} && <div className="text-sm text-green-700">{msg}</div>
      </div>
    </div>
  )}
);
```

```
<input className="w-full p-2 border rounded mb-2" placeholder="CODE" value={code}
onChange={e=>setCode(e.target.value.toUpperCase())} />
<div className="flex gap-2 justify-end">
  <button className="px-3 py-1 border rounded" onClick={()=>setOpen(false)}>Cancel</button>
  <button className="px-3 py-1 bg-green-600 text-white rounded" onClick={join}>Join</button>
</div>
</div>
</div>
  )}
</>
);
}
```


Demonstration

Digital Classroom

Login

Register

Login

ugesht0411@gmail.com

Login

Sign in with Google

Don't have an account? [Register](#)

Login page

Digital Classroom

Login

Register

Register

Full name

Email

Password

☒ Student ☐ Teacher

Register

Register page



Teacher's Dasboard

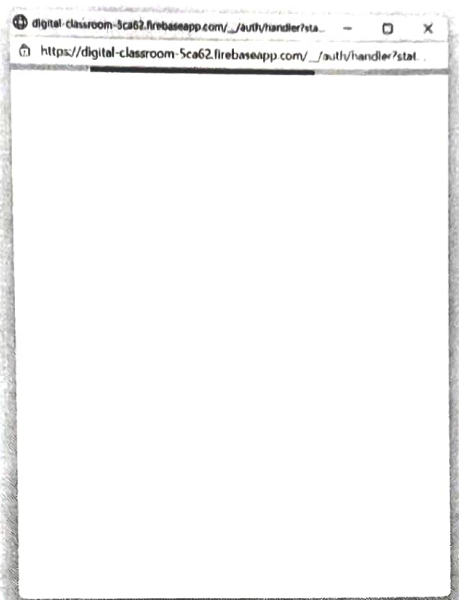


Student's Dashboard

Digital Classroom

Login

Register



Signup with google

Digital Classroom STUDENT

Dashboard

Profile

Logout

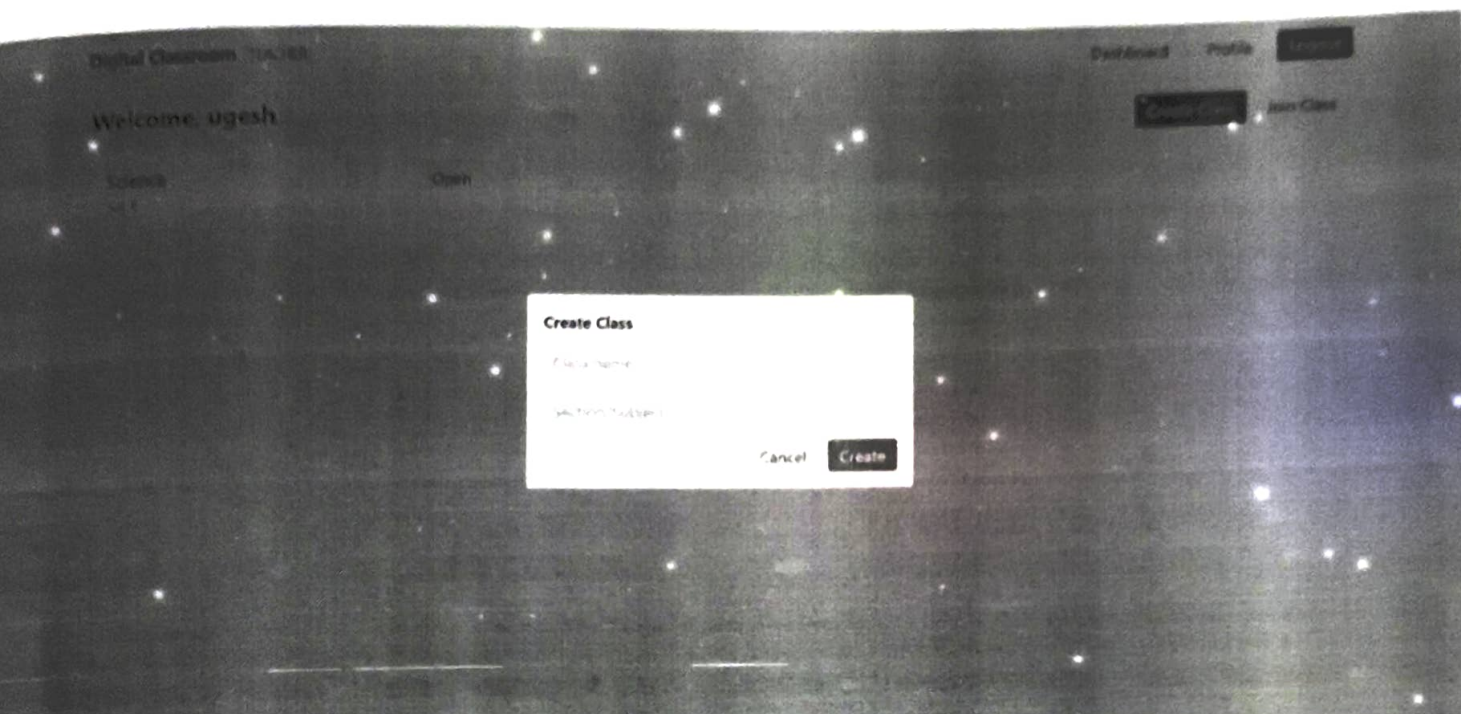
Science

Code: R1WJ50

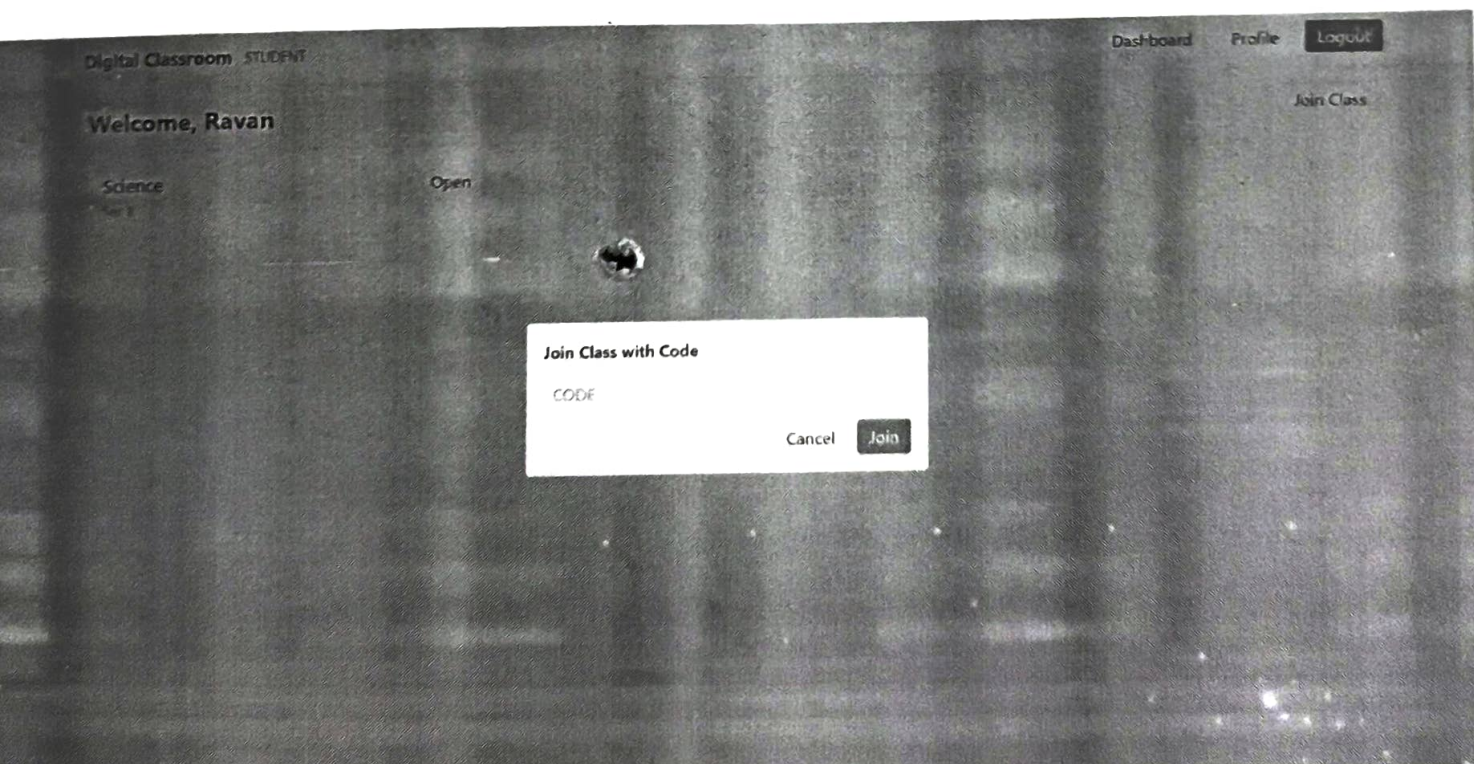
ugesh • 3/10/2025, 9:05:56 pm

Hi guys how are you doing

Classroom posts



Creating class



Joining to a class